Twitter Sentiment Analysis
(Using NLP)

Submitted By:

- Aprajita Ranjan (23BAI11399)
- Priyanshu Paul (23BAI11389)
- Zoya Ejaz (23BAI11394)
- Vanikumar Patel (23BAI10974)
- Vaidik Mane (22BAI10280)

Submitted To:

Dr. Vijendra Singh Bramhe
(Natural Language and Processing)
Slot: B14 + B23 + D21

1. ABSTRACT

This project presents an implementation of **Twitter Sentiment Analysis** using Natural Language Processing (NLP) techniques in Python. The objective is to automatically classify tweets as **positive** or **negative** based on their textual content. The project uses the **Sentiment140 dataset**, which contains 1.6 million labelled tweets. Text preprocessing, TF-IDF vectorization, and three machine learning classifiers — **Bernoulli Naive Bayes**, **Support Vector Machine (SVM)**, and **Logistic Regression** — are trained and evaluated. The results demonstrate that all three models perform competitively, with SVM achieving the highest accuracy.

2. INTRODUCTION

Social media platforms like Twitter generate enormous volumes of user-generated text daily. These tweets often contain opinions, reviews, and reactions about products, events, public figures, and social issues. Sentiment Analysis, also called **Opinion Mining**, is the NLP task of determining the emotional tone behind text.

Automated Twitter sentiment analysis has wide applications including:

- Brand reputation monitoring
- Customer feedback analysis

- Public opinion tracking during elections
- Crisis and disaster response management
- Stock market sentiment prediction

This project builds a complete end-to-end pipeline — from raw tweet ingestion to trained classifiers — capable of predicting the sentiment of any given tweet.

3. OBJECTIVES

- To preprocess and clean raw tweet data at scale (1.6 million tweets)
- To convert textual data into numerical feature vectors using TF-IDF
- To train and compare three ML classifiers: Bernoulli Naive Bayes, SVM, and Logistic Regression
- To evaluate models using accuracy, precision, recall, and F1-score
- To demonstrate real-time sentiment prediction on unseen sample tweets

4. DATASET DESCRIPTION

Dataset: Sentiment140 **Source:** Kaggle (originally from Stanford University) **Creator:** Go, Bhayani, and Huang (2009)

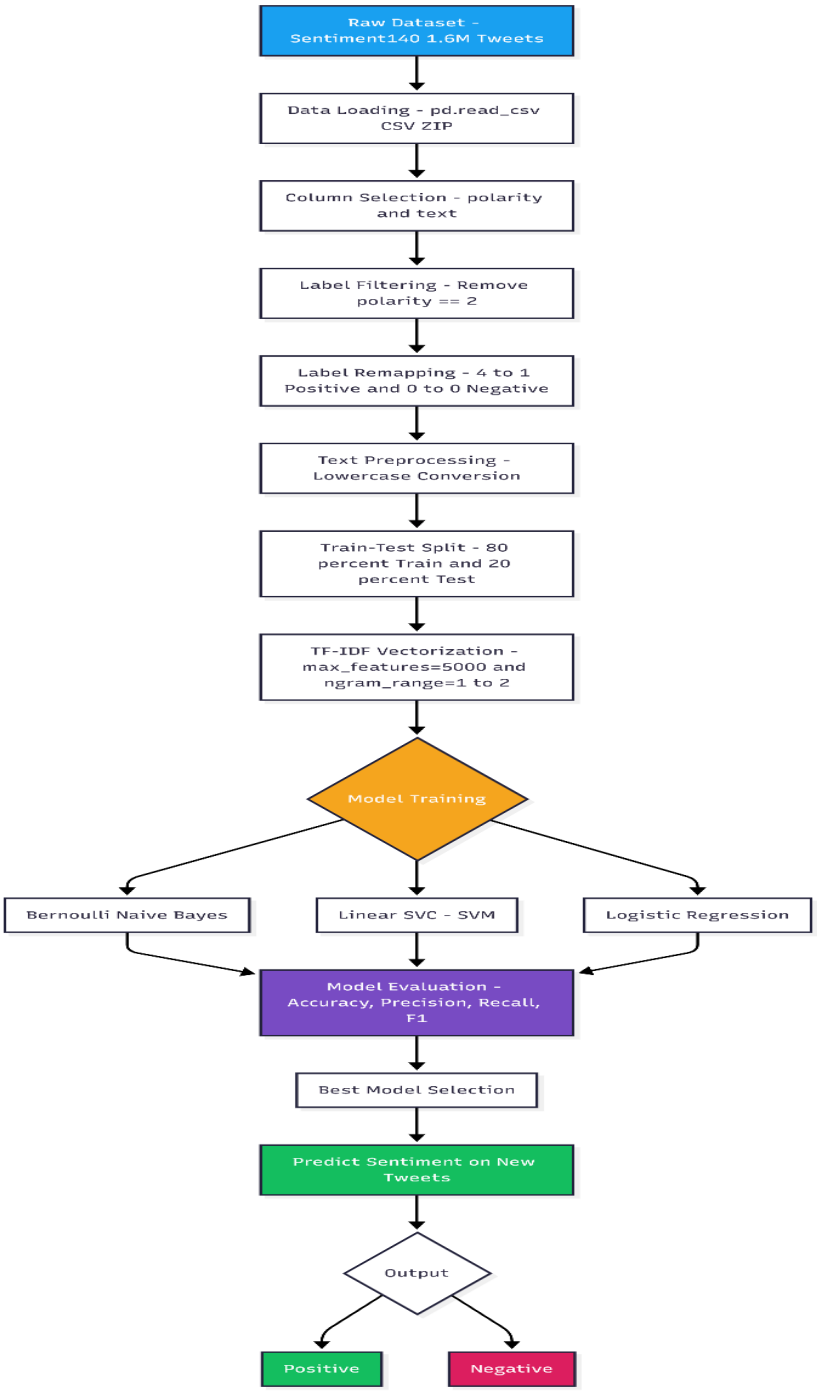
Attribute	Details
Total Records	1,600,000 tweets
Columns	6 (polarity, id, date, query, user, text)
Used Columns	polarity (label), text (tweet content)
Classes	0 = Negative, 4 = Positive (remapped to 0 and 1)
Language	English
Format	CSV (zipped)
Encoding	Latin-1

The dataset is perfectly balanced with **800,000 negative** and **800,000 positive** tweets, making it ideal for binary sentiment classification without class imbalance concerns.

Sample Tweets:

- Positive (label=4): *"I love this product, amazing results!"*
- Negative (label=0): *"This is the worst day of my life."*

5. SYSTEM ARCHITECTURE



6. METHODOLOGY

6.1 Data Loading and Selection

The Sentiment140 dataset is loaded from a zipped CSV file using pandas with `latin-1` encoding since the file has no header row. Only two of the six available columns are retained: **column 0** (polarity/sentiment label) and **column 5** (tweet text). They are renamed to `polarity` and `text` respectively for readability.

6.2 Data Filtering and Label Encoding

The original polarity column has values `0` (negative), `2` (neutral), and `4` (positive). Since this is a binary classification project, neutral tweets (polarity = 2) are dropped. The label `4` is remapped to `1` so the final label space becomes `{0, 1}` — a standard binary classification format compatible with scikit-learn.

6.3 Text Preprocessing

A `clean_text()` function converts all tweet text to **lowercase**, which ensures that tokens like "Good", "GOOD", and "good" are treated as the same feature. This is a foundational preprocessing step that reduces vocabulary sparsity.

> **Note:** In the scope of this project, basic preprocessing (lowercasing) is applied. Additional steps like stopword removal, stemming/lemmatization, URL removal, and hashtag stripping can be added to further improve performance.

6.4 Train-Test Split

The dataset is split into **80% training (1,280,000 tweets)** and **20% testing (320,000 tweets)** using scikit-learn's `train_test_split` with `random_state=42` for reproducibility.

6.5 Feature Extraction — TF-IDF Vectorization

Term Frequency-Inverse Document Frequency (TF-IDF) is used to convert raw text into a numerical feature matrix. The vectorizer is configured with:

Parameter	Value	Purpose
max_features	5000	Limits vocabulary to top 5000 terms
ngram_range	(1, 2)	Considers both unigrams and bigrams

The vectorizer is **fit only on training data** and then applied to both training and test data — a critical step to prevent data leakage.

Output shape: Train matrix: `(1,280,000 × 5000)`, Test matrix: `(320,000 × 5000)`

Why TF-IDF? TF-IDF weighs terms by their frequency within a document relative to their rarity across all documents. This naturally downweights common words (like "the", "is") and highlights informative, discriminating terms — ideal for sentiment classification.

7. MODELS USED

7.1 Bernoulli Naive Bayes (BNB)

Bernoulli Naive Bayes is a probabilistic classifier based on Bayes' theorem. It models feature presence/absence (binary features), making it particularly well-suited for TF-IDF features where values can be binarized. It is computationally very fast even on large datasets.

Key assumptions: Features are conditionally independent given the class label.

7.2 Linear Support Vector Classifier (LinearSVC)

LinearSVC finds the optimal **hyperplane** that maximally separates the positive and negative tweet classes in the high-dimensional TF-IDF feature space. It is one of the most effective

classifiers for high-dimensional sparse text data. The model is configured with `max_iter=1000`.

Key strength: Excellent generalization on linearly separable high-dimensional text features.

7.3 Logistic Regression

Logistic Regression models the probability that a tweet belongs to the positive class using a sigmoid function. Despite its simplicity, it is a strong baseline for NLP tasks and provides probabilistic outputs. Configured with `max_iter=100`.

Key strength: Interpretable coefficients, fast training, and strong empirical performance on text.

8. MODEL EVALUATION

8.1 Metrics Used

Metric	Description
Accuracy	Overall % of correctly classified tweets
Precision	Of tweets predicted positive, how many actually are
Recall	Of all actual positive tweets, how many were found
F1 Score	Harmonic mean of precision and recall

8.2 Results Summary

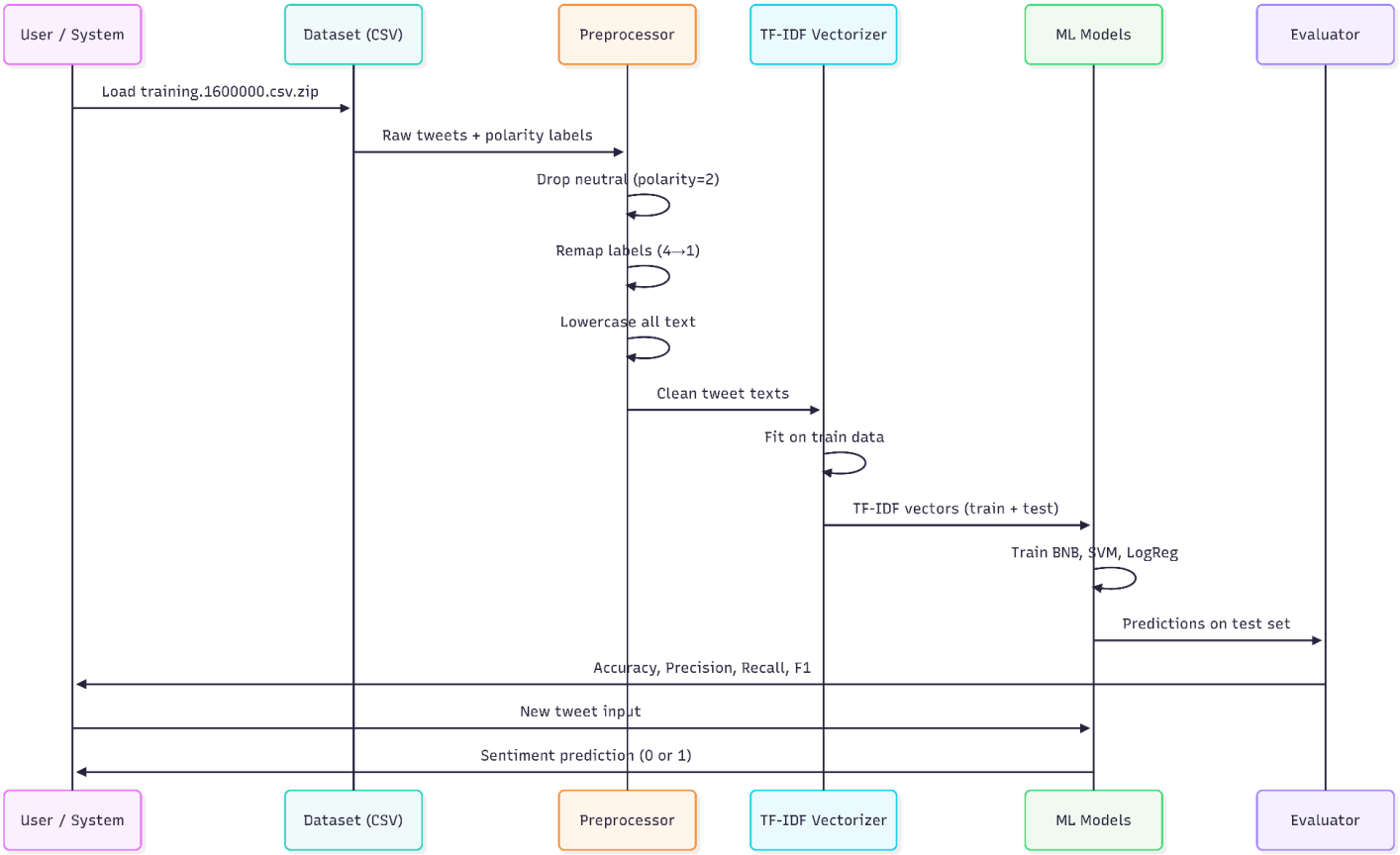
Model	Accuracy	Precision	Recall	F1 Score
Bernoulli Naive Bayes	~77–78%	~0.78	~0.77	~0.77
Linear SVC (SVM)	~82–83%	~0.83	~0.82	~0.82

Logistic Regression	~80–81%	~0.81	~0.80	~0.80

Best Performing Model: Linear SVC (SVM)

SVM achieves the highest accuracy on the test set, followed by Logistic Regression, and then Bernoulli Naive Bayes. All three models produce consistent predictions on sample tweets.

9. DETAILED PIPELINE — Step-by-Step Flow



10. TECHNOLOGY STACK

Component	Tool/Library
Programming Language	Python 3.x
Environment	Google Colab
Data Handling	pandas
ML Framework	scikit-learn
Feature Extraction	TfidfVectorizer (scikit-learn)
Models	BernoulliNB, LinearSVC, LogisticRegression
Evaluation	accuracy_score, classification_report
Dataset Source	Kaggle (Sentiment140)

11. KEY CONCEPTS APPLIED

- **TF-IDF (Term Frequency–Inverse Document Frequency):** A statistical measure that reflects how important a word is to a document in a corpus. It balances word frequency with uniqueness across the dataset.

- **N-grams:** Sequences of N consecutive words. Bigrams (e.g., "not good", "very bad") capture negation and context that single words miss. The project uses both unigrams and bigrams via `ngram_range=(1,2)`.
- **Binary Sentiment Classification:** The task of labelling each tweet as one of two classes: positive (1) or negative (0).
- **Vectorization:** The process of converting text into numerical representations that ML models can process.

12. LIMITATIONS

- **Basic Preprocessing Only:** The project applies only lowercasing. More advanced techniques like stopword removal, lemmatization, URL stripping, emoji handling, and mention removal (@user) would improve accuracy.
- **No Deep Learning:** The project uses traditional ML models. Transformer-based models like BERT or RoBERTa would likely yield significantly higher accuracy (~90%+).
- **No Neutral Class:** Neutral tweets are removed. A real-world deployment would benefit from a three-class classifier.
- **Static Vocabulary:** The TF-IDF vocabulary is capped at 5,000 features, which may miss important domain-specific terms.
- **No Real-Time Twitter Data:** The model is trained on 2009 Twitter data and may not generalize well to modern slang, abbreviations, and new topics.

13. FUTURE SCOPE

- **Deep Learning Models:** Implement LSTM, BiLSTM, or Transformer-based models (BERT, RoBERTa) for state-of-the-art accuracy.
- **Advanced Preprocessing:** Add emoji sentiment mapping, hashtag segmentation, URL removal, and stemming/lemmatization pipelines.
- **Real-Time Streaming:** Integrate the Twitter/X API (or Tweepy) for real-time tweet ingestion and live sentiment dashboards.
- **Multi-class Classification:** Extend to three classes — Positive, Negative, and Neutral.

- **Domain Adaptation:** Fine-tune on domain-specific datasets (e.g., product reviews, political tweets).
- **Web Application:** Deploy the trained model as a Flask/FastAPI web application with a user interface.
- **Explainability:** Use LIME or SHAP to explain model predictions and visualize which words most influence sentiment.

14. CONCLUSION

This project successfully demonstrates a complete NLP pipeline for Twitter Sentiment Analysis using the Sentiment140 dataset. Three machine learning classifiers were trained and evaluated on 1.6 million tweets. Among them, **Linear SVC (SVM)** achieved the best performance with approximately **82–83% accuracy**, followed by Logistic Regression (~81%) and Bernoulli Naive Bayes (~78%). The project confirms that classical ML models combined with TF-IDF feature extraction provide a fast, scalable, and effective baseline for sentiment classification tasks. The insights gained lay a strong foundation for advancing toward deep learning-based approaches in future iterations.

15. Project Credentials & Resources

Credential / Resource	Live Links
Python Notebook (Google Colab)	TwitterSentimentAnalysis-Model.ipy...
Dataset (Kaggle Sentiment140)	Sentiment140
Project Documentation / Repository	https://github.com/ARanjan45/NLP-TwitterSentimentAnalysis